GATE
CS & DA

Artificial Intelligence

Adversarial Search

Lecture No. 3

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

Topic α, β pruning

Topic Minimax

Topic

Topic

Topic

Turn on Slide map

Topics to be Covered

- Topic — α, β pruning
- Topic — Mini max
- Topic
- Topic
- Topic

Plan → Schedule

maximum Utilisation

"Either you run the day or the day runs you."

—Jim Rohn.

Adversarial Search

- Adversarial Search: A search in scenarios where multiple agents (players) have conflicting objectives, and each agent's success is often measured against the other's failure. Unlike traditional search problems (like pathfinding), adversarial search considers the presence of an opponent actively working against the agent's goal.

Zero Sum game → One player will win other will loose.

Minimax Algorithm: → • Both players play optimally. DFS

- The Minimax algorithm is a recursive, decision-making algorithm used in two-player games, particularly zero-sum games where one player's gain is another player's loss. It systematically explores the game tree to determine the optimal move for a player, assuming that the opponent also plays optimally. →
- It is a specialized Search Algo that returns optimal Sequence move for a player in a Zero-Sum Game.
- Recursive/ Back tracking algo which is used in decision making and Game theory and uses recursion to Search through Game Tree.
- Algo Computes minimax decision for current state.
- We have 2 players : max and min player. Max player select the maximum value and min player select the minimum value
- Depth-First Search Algo is used for exploration of Complete Game Tree.
- Used in Chess, Tic Tac Toe and other 2 player Games

Minimax Algorithm:

- Initial values for Max and Min player : + infinity and – Infinity
- This is the backtracking algorithm
- Complete and optimal solution
- Time and Space complexity
- In Minimax algorithm we use DFS

β α

• minimax ⇒ normal DFS
all node visit

$O(b^d)$ ← TC →
$O(bd)$ ← SC →
Same as DFS

Minimax Algorithm:

- Conceptual Overview Two Players: → what two players want.
  - Often referred to as the "Maximizer" and the "Minimizer. → Player name
  - "Maximizer: Tries to maximize their score. → Score = max(Child score)
  - Minimizer: Tries to minimize the Maximizer's score (which is equivalent to maximizing their own score if you flip the sign). → Score = min(Child score)
- Game Tree: A tree structure representing all possible moves (edges) from a given game state (nodes).
- Root Node: Represents the current state of the game.
- Leaf Nodes: Represent terminal states where the game ends (e.g., win, loss, or draw).

Minimax Algorithm:

Minimax Principle

- The algorithm assumes that both players play optimally.
- It simulates all possible moves from the current state down to terminal states, then back-propagates the values up the tree to make the best decision at the root.

Minimax Algorithm:

- Time Complexity:
- The time complexity of the basic Minimax algorithm depends on the branching factor b (the average number of legal moves at each point) and the depth d (the depth of the game tree).
- Since Minimax explores the entire tree, it evaluates all possible moves up to the depth d.
- Time Complexity: $O(b^d)$

Minimax Algorithm:

- Space Complexity:
- Same as DFS algorithm
- Time Complexity: $O(b^d)$

(optimized version of minimax algo)

Alpha-Beta Pruning

- Alpha-beta pruning is an optimization technique for the minimax algorithm, commonly used in decision-making for two-player games like chess, tic-tac-toe, and other adversarial games. The main purpose of alpha-beta pruning is to reduce the number of nodes evaluated by the minimax algorithm, thereby speeding up the decision-making process without affecting the final decision.

⇒ * 2 Players
* min/max
* Both play optimally
* DFS
* Backtracking

* Pruning
* Complete & optimal

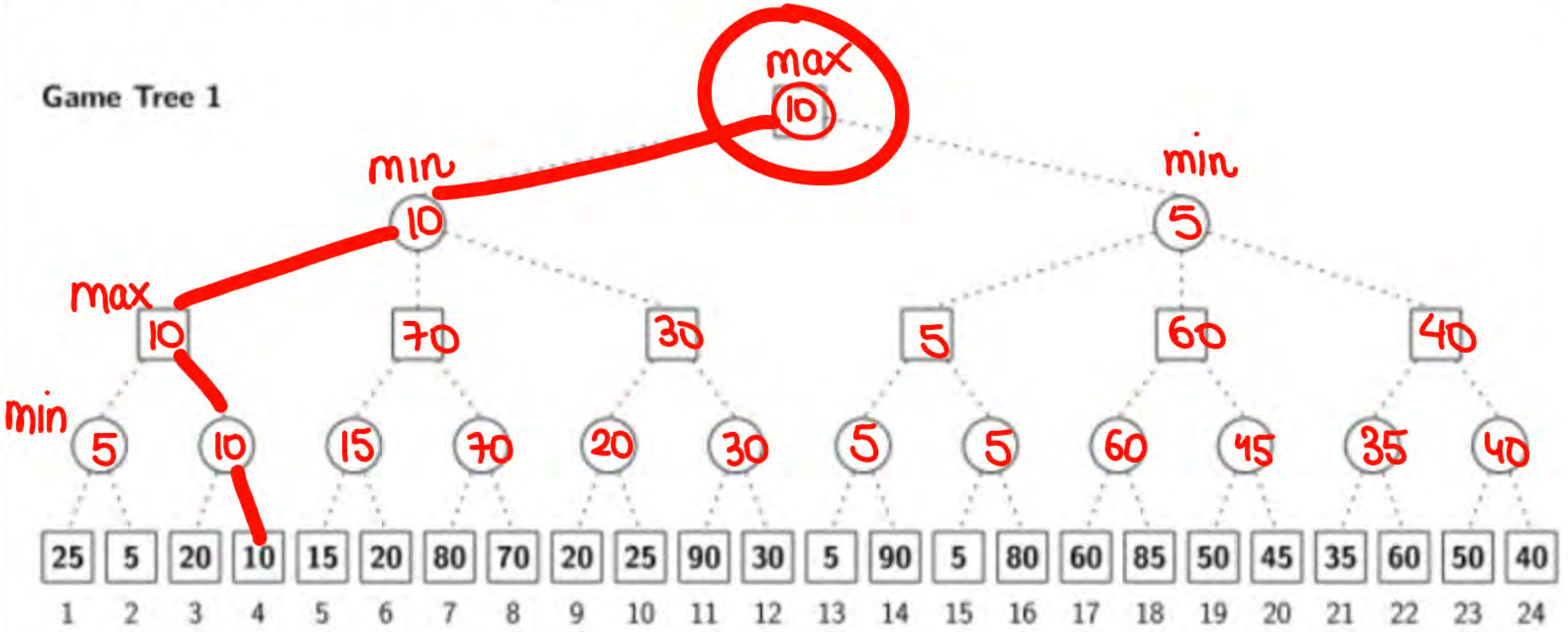
Alpha-Beta Pruning

- This is advanced version of minimax algorithm
- Because minimax algorithm has to explore all the nodes thus time complexity was $O(b^d)$
- But in this new algorithm we explore less number of nodes
- Alpha : is for max player
- Beta : is for min player

9) What is the MinMax value of the game?

The figure shows a 4-ply game tree with evaluation function values at the horizon. The nodes in the horizon carry reference numbers (sequence numbers going left to right) at the bottom. Use these reference numbers when you want to enter a list of nodes. Where necessary assume top-down and left-to-right node order. Use this game tree to answer the following questions.

Game Tree 1



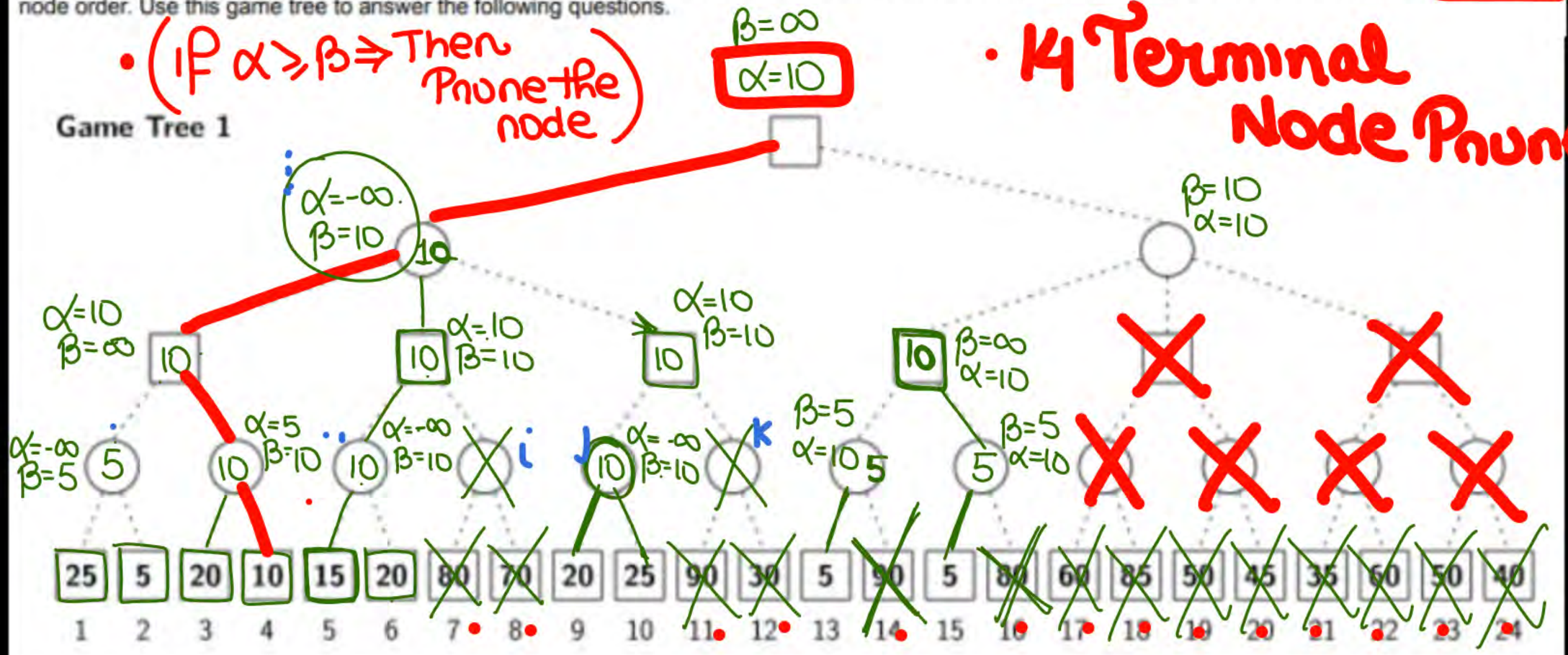
12) How many nodes are pruned (unexplored) by AlphaBeta algorithm? Count only the horizon nodes.

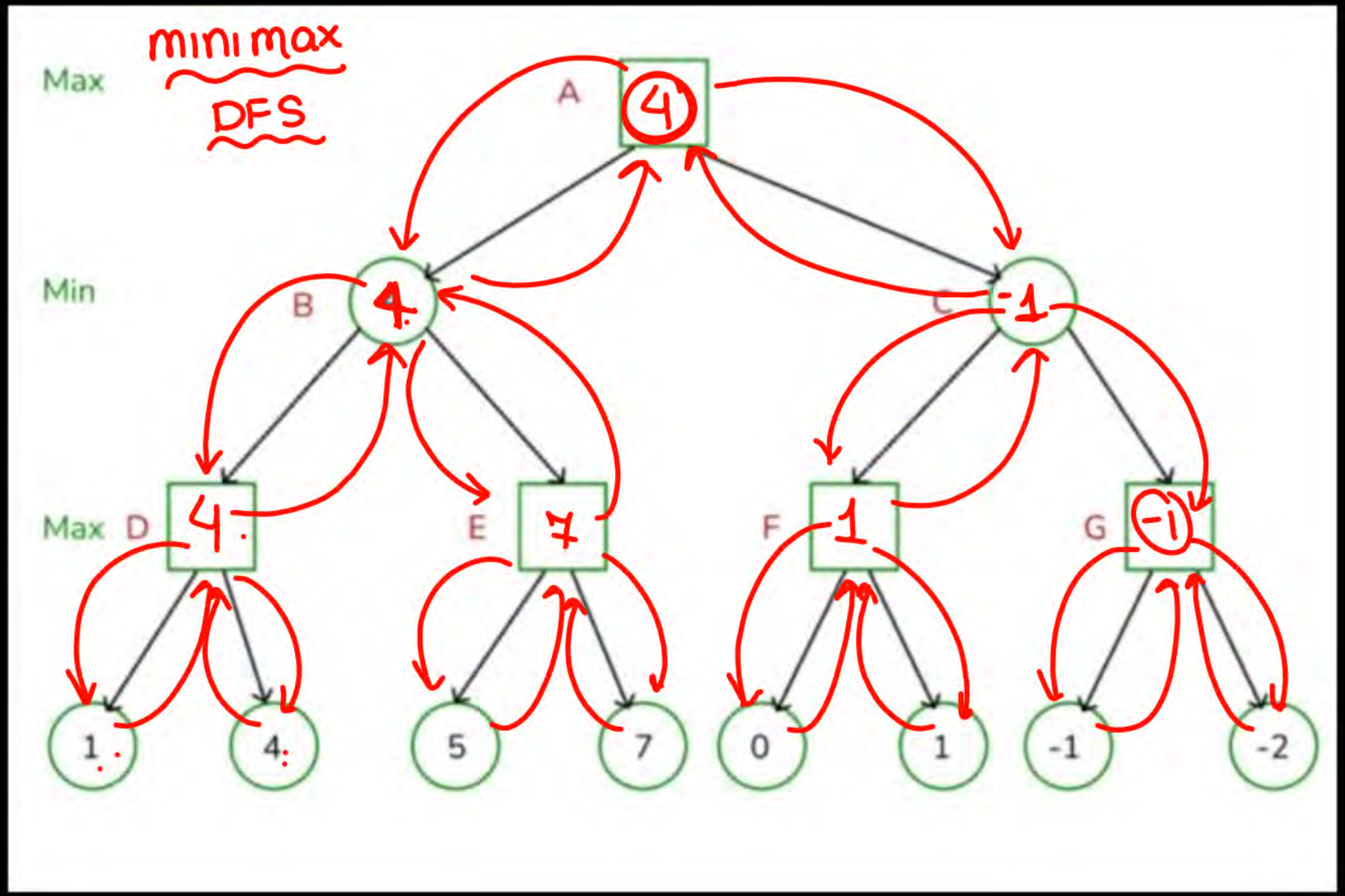
The figure shows a 4-ply game tree with evaluation function values at the horizon. The nodes in the horizon carry reference numbers (sequence numbers going left to right) at the bottom. Use these reference numbers when you want to enter a list of nodes. Where necessary assume top-down and left-to-right node order. Use this game tree to answer the following questions.

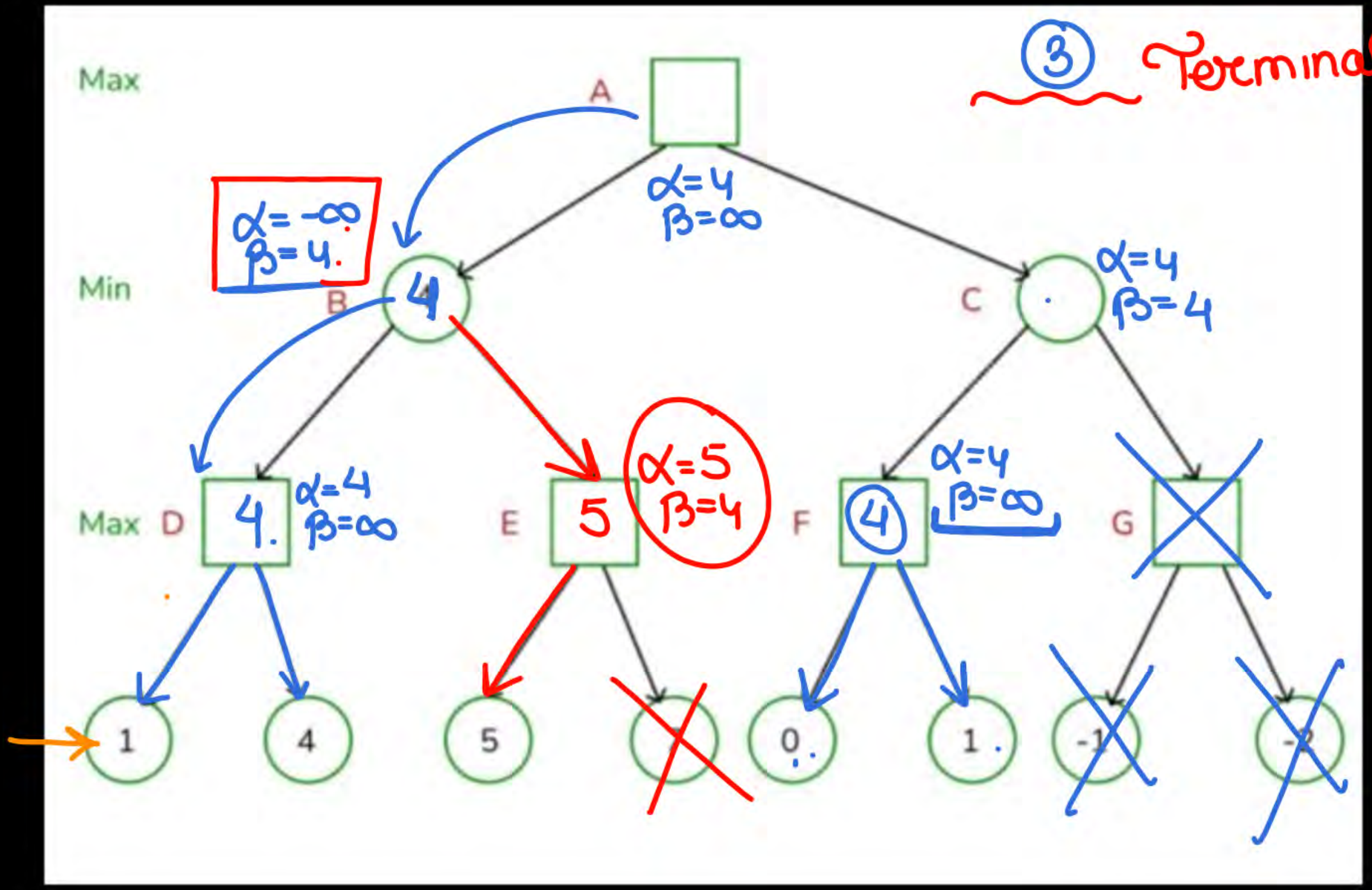
• (If $\alpha \geq \beta \Rightarrow$ Then Prune the node)

• 14 Terminal Node Pruned.

Game Tree 1




minimax
DFS
Max
A
4
Min
B
4
C
-1
Max
D
4
E
7
F
1
G
-1
1
4
5
7
0
1
-1
-2



3 Terminal node Prune

Alpha-Beta Pruning – The complete procedure

- The procedure for solving any general alpha-beta pruning problem can be broken down into a set of steps that are followed to systematically explore the game tree while minimizing the number of evaluations. Below is the step-by-step procedure for solving any alpha-beta pruning problem:
- 1. Initialization:
  - Start at the root of the game tree.
  - Set initial values for Alpha to -∞ (lowest possible) and Beta to ∞ (highest possible).

    updated @max player
  - Alpha: Keeps track of the maximum lower bound that the maximizing player (MAX) is assured of.
  - Beta: Keeps track of the minimum upper bound that the minimizing player (MIN) is assured of.

Alpha-Beta Pruning – The complete procedure

- 2. Traverse the Tree Depth-First: Use a depth-first search (DFS) traversal to explore the tree. Start from the root node and proceed down one branch, only moving to another branch if necessary.
- 3. Update Alpha and Beta:
  - At each node, based on whether it is a MAX or MIN node:
  - MAX node: ⇒ update α
  - Maximizing player (goal is to maximize the utility value).
  - Start with the current value of Alpha.
  - At each child, update Alpha with the maximum value of its children.
  - Update: Alpha = max(Alpha, child value).
  - MIN node:
  - Minimizing player (goal is to minimize the utility value).
  - Start with the current value of Beta.
  - At each child, update Beta with the minimum value of its children.
  - Update: Beta = min(Beta, child value).

Alpha-Beta Pruning – The complete procedure

- 4. Prune When Alpha ≥ Beta:
- While traversing the tree, check if Alpha ≥ Beta at any point: Alpha ≥ Beta means that the current branch will not affect the outcome because the maximizing player can already guarantee a better result or the minimizing player can avoid a worse outcome.
- When this condition holds, you can prune the remaining branches below that node, as further exploration is unnecessary.
- 5. Backtrack and Propagate Values:
- Once a leaf node is evaluated, propagate the computed value up to its parent node:
- At MAX nodes, propagate the maximum value from the children.
- At MIN nodes, propagate the minimum value from the children.

Alpha-Beta Pruning – The complete procedure

- 6. Repeat Until All Nodes Are Visited or Pruned:
- Continue traversing the tree until: You have visited all necessary nodes, or Pruning has eliminated further exploration.

α, β pruning

→ TC ⇒ (worst case) all node need to be visited ⇒ $O(b^d)$

SC ⇒ (DFS) $O(b \cdot d)$

Alpha-Beta Pruning – The complete procedure

- Space Complexity:
- Like Minimax, the space complexity in Alpha-Beta pruning is determined by the depth of the recursion stack.
- Space Complexity: $O(\quad)$,

done

1. Which of the following statements is true for the Minimax algorithm in adversarial search?

- (A) It only works for one-player games.
- (B) It minimizes the outcome of a game for both players.
- (C) It assumes that both players play optimally.
- (D) It evaluates only a subset of the game tree.

2. In a Minimax algorithm, how is the value of a node computed in a game tree?

- (A) The sum of values of its children nodes.
- (B) The average of values of its children nodes.
- (C) The maximum or minimum value from its children, depending on whether it's a Maximizer or Minimizer node.
- (D) The product of values of its children nodes.

3. What is the time complexity of the Minimax algorithm in a game tree with branching factor **b** and depth **d**?

- (A) $O(b \cdot d)$
- (B) $O(b + d)$
- (C) $O(b^d)$ ← TC
- (D) $O(b^{d/2})$

4. In Alpha-Beta pruning, pruning occurs when:

- (A) A node's value is greater than or equal to α for a Minimizer.
- (B) A node's value is less than or equal to β for a Maximizer.
- (C) A node's value exceeds β for a Maximizer or is lower than α for a Minimizer. → $\beta \leq \alpha$
- (D) A node's value is between α and β.

$\alpha \geq \beta$

1. Which of the following are benefits of Alpha-Beta Pruning over the Minimax algorithm?

- (A) It reduces the time complexity in the best-case scenario.
- (B) It guarantees optimal solutions even if some branches are pruned.
- (C) It guarantees faster execution than Minimax in all cases.
- (D) It reduces space complexity compared to Minimax.

2. Which of the following statements are true for the Alpha-Beta pruning algorithm?

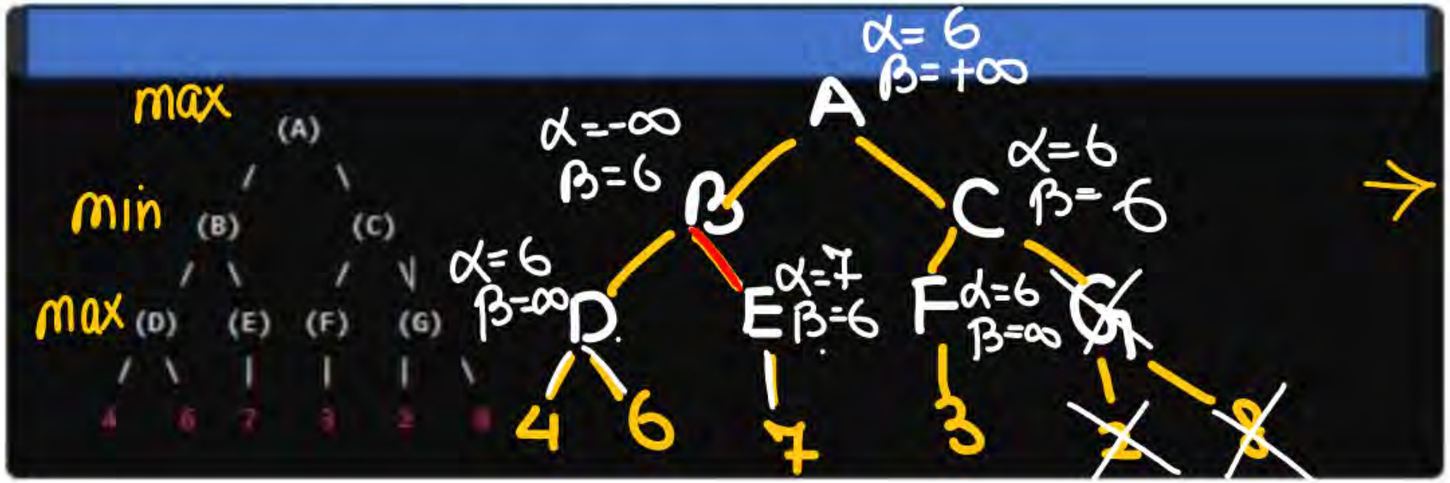
- (A) It evaluates every node in the game tree.
- (B) It requires the tree to be searched depth-first.
- (C) It reduces the time complexity of Minimax in the best case to $O(b^{d/2})$.
- (D) Pruning depends on the order of node exploration.

Remember.

best Case
TC $O(b^{d/2})$

2. Terminal node Pruned

Consider the following game tree where the values are terminal scores:



Subtree below G is pruned

Question:

Use the Minimax algorithm with Alpha-Beta pruning to compute the value of the root node **A**.

Specify which branches will be pruned.

THANK - YOU